

U
N
S
A



Universidad Nacional de San Agustín de Arequipa
Facultad de Producción y Servicios
Departamento académico de Ingeniería de Sistemas e Informática
Escuela Profesional de Ingeniería de Sistemas

Caso de análisis
Patrón Publish/Subscribe

Curso
Sistemas Distribuidos

Docente
Ing. Maribel Molina Barriga

Estudiantes
Almanza Mamani Edgar Raul
Chambi Velasquez, Rommel Abraham
Huayhua Hillpa, Yourdyy Yossimard
Perez Huamani Jeremy Joshua
Sucle Suca Michael Benjamin
Zuñiga Villacorta Peter Sebastian

2026



Patrón Publish/Subscribe

Índice:

Patrón Publish/Subscribe.....	1
Índice:.....	1
Definición:.....	1
Ventajas principales.....	1
Limitaciones clave.....	2
¿Por qué es óptimo para notificaciones universitarias?.....	2
Diseño Arquitectónico del Sistema.....	2
Topología de la Red y Componentes del Sistema.....	2
Ciclo de Vida y Componentes del Sistema.....	3
Funcionamiento.....	4
Referencias.....	6

Definición:

Patrón de mensajería asíncrona donde los publicadores envían mensajes a un broker por temas (*topics*), y los suscriptores reciben solo los mensajes de los temas que les interesan. No se conocen entre sí.

Ventajas principales

- Alto desacoplamiento: Publicadores y suscriptores operan de forma independiente sin necesidad de conocer la identidad o ubicación de los demás.
- Escalabilidad horizontal: Se pueden agregar nuevos publicadores o suscriptores sin modificar los componentes existentes, distribuyendo la carga eficientemente.
- Comunicación uno-a-muchos: Un solo evento llega simultáneamente a todos los suscriptores interesados, ideal para difusión masiva.
- Tolerancia a fallos parciales: Si un suscriptor falla, el resto del sistema sigue funcionando sin afectación.
- Integración heterogénea : Permite que sistemas escritos en diferentes lenguajes y plataformas se comuniquen sin problemas.

Limitaciones clave

- Complejidad añadida: La inclusión de un broker intermedio introduce más componentes a gestionar y mantener.



- Sin garantía de entrega nativa: El patrón básico no asegura que todos los mensajes lleguen a todos los suscriptores, especialmente si están desconectados.
- Orden no garantizado: Los mensajes pueden llegar en orden distinto al de publicación a menos que se implementen mecanismos adicionales.
- Posible duplicación de mensajes: Algunas implementaciones pueden enviar el mismo mensaje múltiples veces, requiriendo suscriptores idempotentes.
- Comunicación unidireccional: El patrón no contempla respuestas; si se necesita dos vías, debe combinarse con otros patrones.

¿Por qué es óptimo para notificaciones universitarias?

Permite difundir un solo evento (cambio de horario, nueva nota, recordatorio) a miles de estudiantes en tiempo real, con suscripción por temas (por carrera, curso, etc.), ahorrando recursos y evitando cuellos de botella.

Diseño Arquitectónico del Sistema

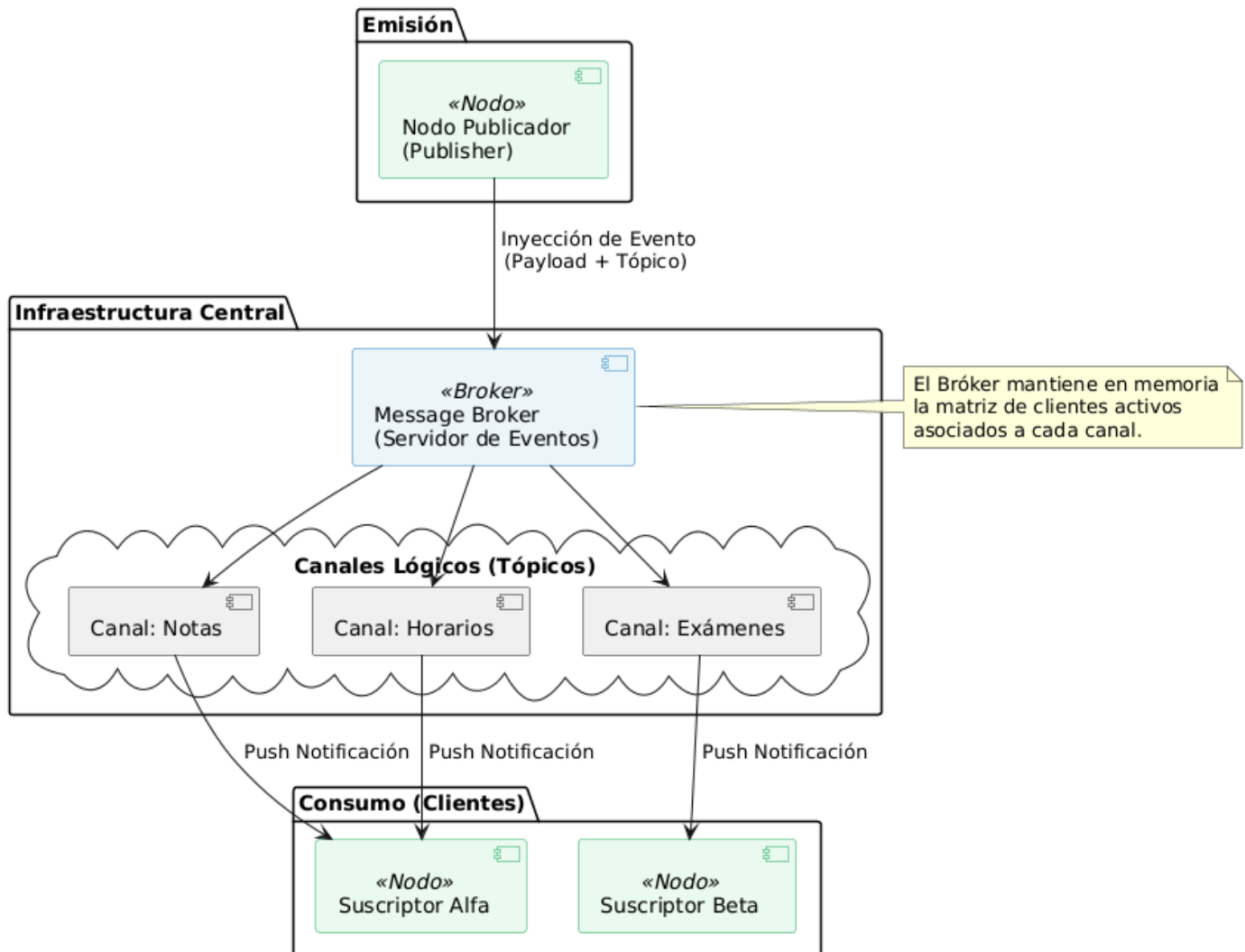
El sistema de alertas universitarias se ha estructurado bajo un enfoque de arquitectura distribuida, implementando el patrón de diseño Publish/Subscribe (Pub/Sub) de topología centralizada. Este modelo reemplaza el paradigma tradicional de comunicación acoplada (Cliente-Servidor directo) por un ecosistema basado en eventos y mediado por un bróker central.

La comunicación se soporta sobre canales bidireccionales y persistentes en la capa de aplicación, garantizando el paradigma Push (empuje de datos) con una latencia mínima.

A. Topología de la Red y Componentes del Sistema

La arquitectura está compuesta por tres roles abstractos interconectados, donde la inteligencia del enrutamiento reside exclusivamente en el núcleo central:

- **Nodo Publicador (Publisher):** Componente emisor encargado de capturar o generar los eventos del entorno (ej. alertas académicas). No posee conocimiento sobre el destino final de los datos ni sobre la cantidad de receptores.
- **Intermediario de Mensajes (Message Broker):** El motor lógico del sistema. Administra el ciclo de vida de las conexiones, mantiene el mapa de suscripciones activas y ejecuta el algoritmo de discriminación y reenvío de paquetes.
- **Nodos Suscriptores (Subscribers):** Dispositivos o aplicaciones cliente finales. Establecen enlaces pasivos permanentes a la espera de que el Bróker les inyecte información relevante según sus intereses previamente registrados.

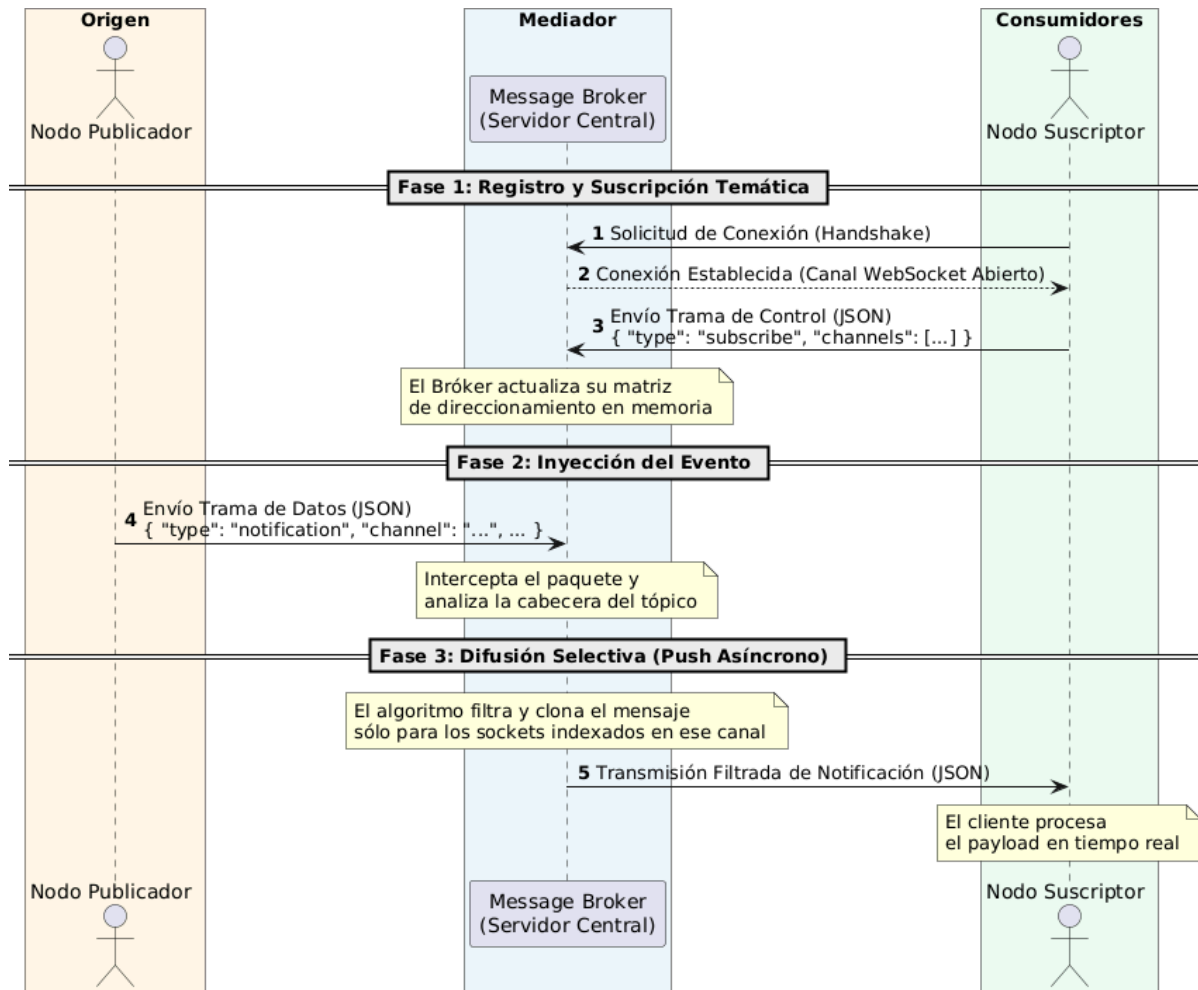


B. Ciclo de Vida del Evento y Flujo de Comunicación

- Fase de Registro y Suscripción Temática:** Los terminales consumidores inician una solicitud de conexión hacia el Bróker Central. Una vez establecido el enlace, el cliente envía un mensaje de control indicando los canales específicos a los que desea asociarse. El Bróker actualiza su matriz de direccionamiento en memoria, vinculando el identificador del cliente con los tópicos solicitados.
- Fase de Inyección del Evento:** El nodo emisor despacha un paquete de datos hacia el Bróker. Este mensaje viaja con una etiqueta obligatoria que define el "tópico" o área temática del aviso. Tras la entrega del paquete, el publicador queda inmediatamente liberado para procesar nuevas tareas.
- Fase de Difusión Selectiva:** El Bróker intercepta el paquete entrante, analiza la cabecera para identificar el canal destino y consulta su matriz de registros. Posteriormente, clona el mensaje y lo distribuye en paralelo únicamente a los enlaces de



los suscriptores indexados en ese canal, omitiendo al resto de la red para optimizar el ancho de banda.



Funcionamiento: <https://github.com/jperezh04/Sistema-notificaciones.git>

Conexion a Servidor

```

Admin@Admin MINGW64 ~/Documents/sd/Sistema-notificaciones/src (master)
$ node server.js

=====
Servidor Publish/Subscribe iniciado
WebSocket escuchando en ws://localhost:8080
Canales: notas, horarios, examenes, tareas, eventos, sistema, general
=====
[CONNECT] Cliente conectado: CLIENT-1779728766672-41
[PUBLISH] NOTIF-0001 | nueva_nota | Canal: notas | Emisor: Almanza
----- ESTADO DEL SERVIDOR -----
Suscriptores conectados : 0
Publicadores conectados : 1
Conexiones totales      : 1
Notificaciones enviadas : 1
Entregas realizadas     : 0
ACK recibidos           : 0
=====
  
```

Caso 1:

Creación de suscriptor Jeremy a Notas, Exámenes y horario.

Panel de Suscriptores
Múltiples sesiones de suscripción en tiempo real

Jeremy
Suscrito a: notas, horarios, exámenes

Gestión de suscriptores
Agrega varios suscriptores, cada uno con su propia conexión WebSocket y boletines independientes.

SUSCRIPTORES ACTIVOS 1

Jeremy Ver boletines Quitar

Suscrito a: notas, horarios, exámenes

notas horarios exámenes

notas Agregar Quitar

NOTIFICACIONES 1

Nueva nota publicada
Ya está disponible la nota de Sistemas Distribuidos.
NOTIF-0001 - notas - Almatz - 12:06:45 p. m.

Panel multi-suscriptor - Servidor: ws://localhost:8080

Caso 2:

Creación de un nuevo suscriptor "Yourdy" para ver solamente la notificación de Horario

Panel de Suscriptores
Múltiples sesiones de suscripción en tiempo real

Yourdy
Suscrito a: horarios

Gestión de suscriptores
Agrega varios suscriptores, cada uno con su propia conexión WebSocket y boletines independientes.

SUSCRIPTORES ACTIVOS 2

Jeremy Ver boletines Quitar

Suscrito a: notas, horarios, exámenes

notas horarios exámenes

notas Agregar Quitar

NOTIFICACIONES 2

Cambio de horario
La clase fue cambiada para otro horario.
NOTIF-0002 - horarios - Almatz - 12:08:46 p. m.

Nueva nota publicada
Ya está disponible la nota de

Yourdy Ver boletines Quitar

Suscrito a: horarios

horarios

notas Agregar Quitar

NOTIFICACIONES 1

Cambio de horario
La clase fue cambiada para otro horario.
NOTIF-0002 - horarios - Almatz - 12:08:46 p. m.

Panel multi-suscriptor - Servidor: ws://localhost:8080

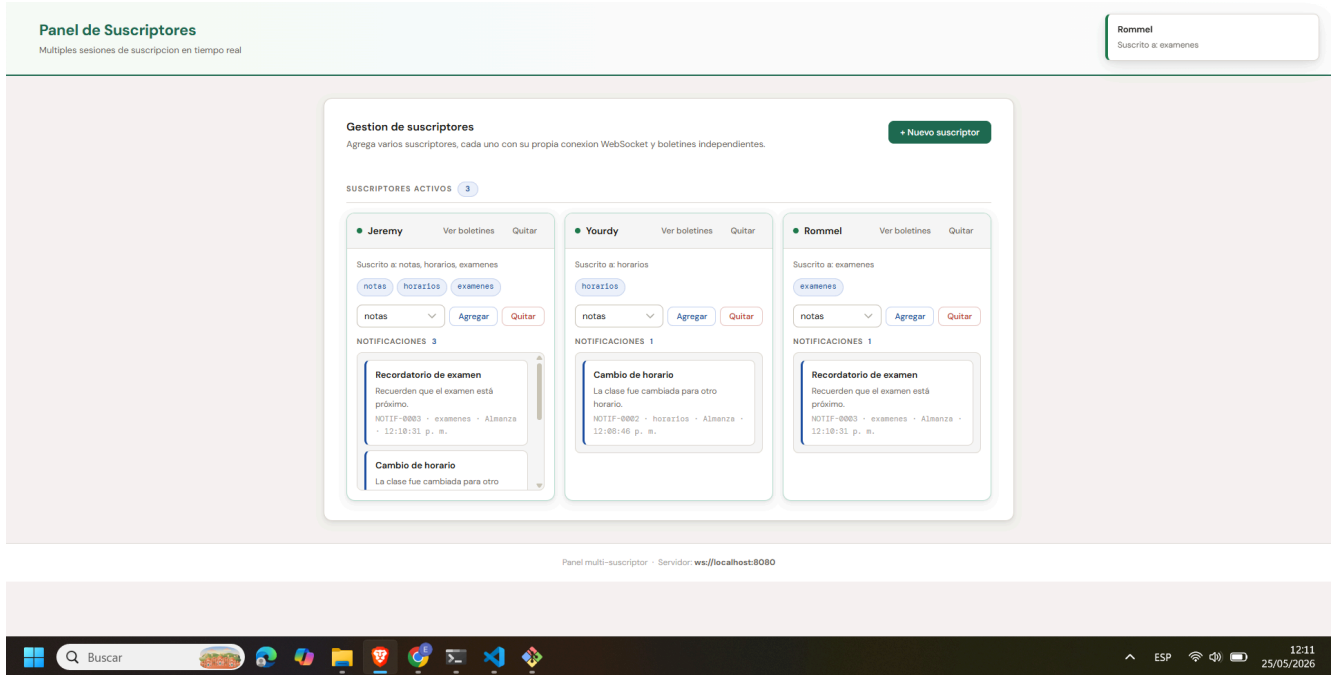


Caso 3:

Creación de Recordatorio de Examen, para ello el mismo Publicador crea la notificación.

Creación de un nuevo suscriptor Rommel para ver solamente la notificación de exámenes

El suscriptor Rommel solo puede ver la notificación de recordatorio de examen ya que solo está suscrito a esa sección.



Referencias

- AWS. (2026). *Publish-subscribe pattern*.
- Microsoft. (2026). *Publisher-Subscriber pattern*.
- Wikipedia. (2024). *Publish-subscribe pattern*.
- Malik, B.H. et al. (2018). *University Notification Subscription System using Amazon Web Service*. IJACSA, 9(5).
- Baeldung. (2024). *Publisher-Subscriber Model*.